Advanced Statistical Inference -- Classification with MCMC

In this lab, we will explore classification tasks using Bayesian methods, specifically Markov Chain Monte Carlo (MCMC). We will implement this technique to classify data points and compare their performance.

In doing so, we will cover:

- Setting up a classification problem
- Implementing Metropolis-Hastings MCMC for classification
  - Evaluating convergence and performance
- Learn a probabilistic programming framework (NumPyro)
  - Review your implementation with a reference library implementation (you should get similar results)

This lab will provide hands-on experience with Bayesian classification methods and help solidify your understanding of MCMC in practical applications.

How to approach this lab:

- Follow the instructions step-by-step.
- Most of the code is provided; your task is to fill in the missing parts and run the code to see the results.
- Experiment with different parameters and settings to observe their effects on the classification performance.
- Use the lecture notes and the course slides to assist you in completing the lab.

Important:

In this lab, we will use JAX for numerical computations. If you haven't used JAX before (and even if you have), use the provided tutorials to get familiar with its syntax and functionalities. Everything that you will need in terms of JAX is covered in the tutorials.

Do not proceed further until you have reviewed the JAX tutorial!

Setup

First, let's import the necessary libraries and set up our environment.

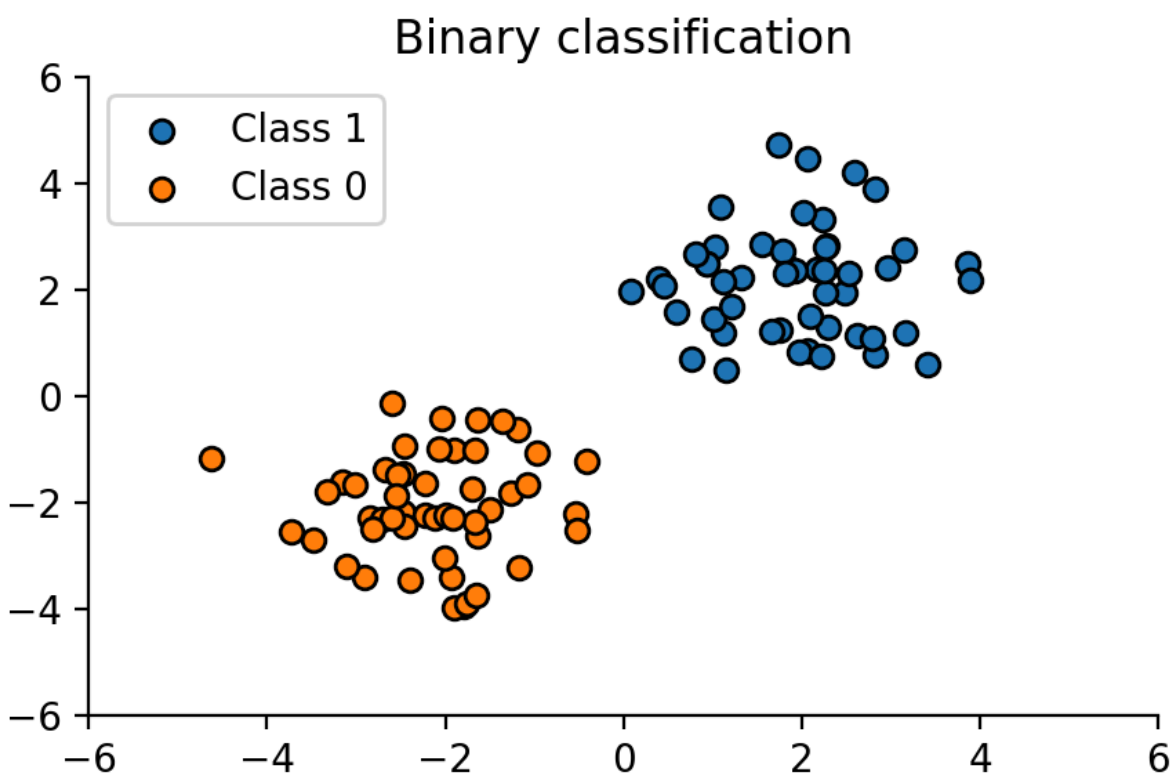
In [1]:
```
import warnings
from functools import partial

import jax
```

```
import jax.numpy as jnp
import matplotlib.pyplot as plt
import numpy as np
from matplotlib import rc

warnings.filterwarnings("ignore")

colab = "google.colab" in str(get_ipython())
rc("figure", **{"dpi": 200})
rc(
    "axes",
    **{"spines.right": False, "spines.top": False, "xmargin": 0.0, "ymargin"
)


def plot_data(X, y, ax):
    """Visualize the training data with different colors for each class.

    Args:
        X: Input features (N, 2) array
        y: Binary class labels (N,) array with values 0 or 1
        ax: Matplotlib axis object to plot on
    """
    mask = y == 1
    config = dict(edgecolor="black", linewidth=1, zorder=10)
    ax.scatter(*X[mask].T, label="Class 1", facecolor="tab:blue", **config)
    ax.scatter(*X[~mask].T, label="Class 0", facecolor="tab:orange", **confi


def get_grid(xlim=(-3.0, 3.0), ylim=None, N=100):
    """Create a 2D grid of points for plotting decision boundaries and poste

    This is useful for evaluating the posterior or predictive probabilities

    Args:
        xlim: Tuple (min, max) for x-axis range
        ylim: Tuple (min, max) for y-axis range. If None, uses xlim
        N: Number of grid points per dimension (creates N x N grid)

    Returns:
        xx, yy: 2D meshgrid arrays for plotting contours
        X_plot: Flattened (N*N, 2) array of all grid point coordinates
    """
    if ylim is None:
        ylim = xlim
    x_grid = np.linspace(*xlim, N)
    y_grid = np.linspace(*ylim, N)
    xx, yy = np.meshgrid(x_grid, y_grid)
    X_plot = np.vstack((xx.flatten(), yy.flatten())).T
    return xx, yy, X_plot
```

Exercise: Load and plot the data

In [4]:
```
from sklearn.datasets import make_blobs
```

```
X, y = make_blobs(n_samples=100, centers=[[-2, -2], [2, 2]], cluster_std=1.0

fig, ax = plt.subplots(figsize=[5, 3])
# @@ COMPLETE @@
plot_data(X, y, ax)
ax.set_title("Binary classification")
ax.legend()
ax.set_xlim(-6, 6)
ax.set_ylim(-6, 6)
plt.show()
```



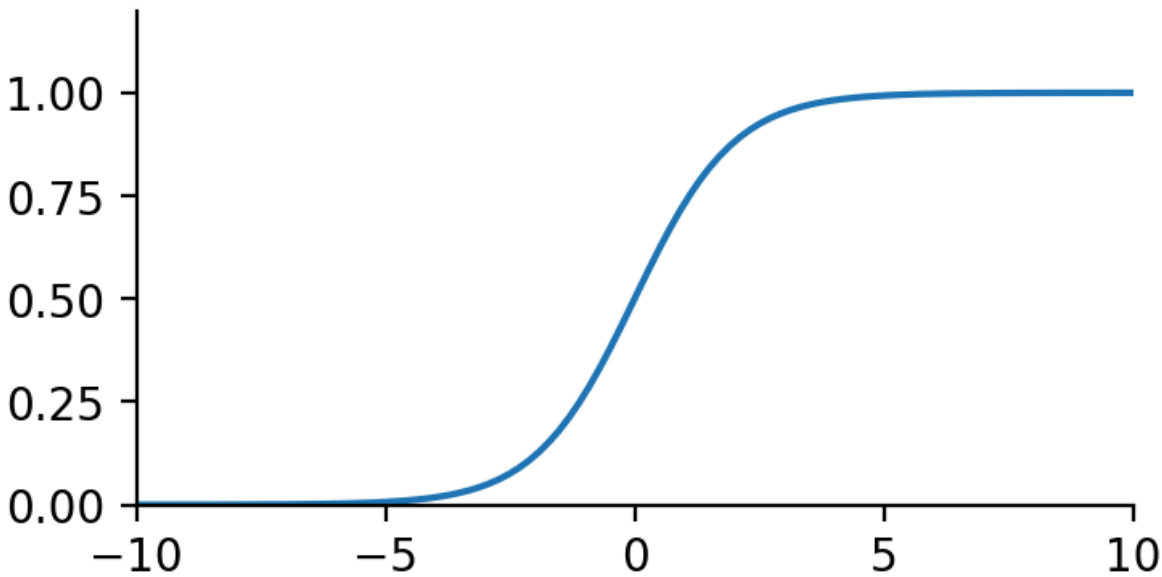
Logistic Regression Setup

For binary logistic regression, we model the probability of class 1 as a function of the inputs through the logistic (sigmoid) function: $h(z) = (1 + \exp(-z))^{-1}$. This maps the unbounded linear predictor $w^T x$ to a valid probability in [0,1].

Exercise: Implement the logistic function and plot it.

In [5]:
```
def logistic(z):
    """Compute the logistic (sigmoid) function.

    The logistic function maps any real-valued input to a probability in [0,

    Args:
        z: Input (can be scalar, array, or batch)

    Returns:
        Probability value(s) in [0, 1] range
```

```
    """
    # @@ COMPLETE @@
    out = 1 / (1 + jnp.exp(-z))
    return out


z = np.linspace(-10, 10, 100)
fig, ax = plt.subplots(figsize=[4, 2])
ax.plot(z, logistic(z))
ax.set_ylim(0, 1.2)
plt.show()
```



Likelihood and Prior

We combine two key components to define the Bayesian model:

Likelihood (Bernoulli): The probability of observing label y given prediction p

$$\log p(y|p) = y \log(p) + (1 - y) \log(1 - p)$$

Prior (Gaussian): We use a zero-mean Gaussian prior to encourage small weights (regularization). This provides a smooth solution and prevents overfitting.

The likelihood and prior terms are defined separately to keep code modular and clear.

In [7]:
```
def bernoulli_logdensity(y, p):
    """Compute the log-likelihood of binary outcomes under a Bernoulli distr

    For binary classification, we assume each label y follows a Bernoulli di
    with probability p of being in class 1.

    The small constant (1e-6) prevents log(0) numerical issues.

    Args:
```

```
        y: Binary labels (0 or 1)
        p: Predicted probabilities in [0, 1]

    Returns:
        Log-likelihood values
    """
    # @@ COMPLETE @@
    out = y * jnp.log(p + 1e-6) + (1 - y) * jnp.log(1 - p + 1e-6)
    return out


def gaussian_logdensity(x, mean=0, var=1):
    """Compute the log-density of a multivariate Gaussian distribution.

    This represents a spherical (isotropic) Gaussian with:
    - mean: zero (or specified mean)
    - covariance: var * I (identity scaled by variance)

    Used as a prior over weights in Bayesian logistic regression.
    The log-density is:
    log p(x) = -0.5 * [(x-mu)^T(x-mu)/sigma^2 + d*log(2*pi*sigma^2)]
    where d is the dimensionality (len(x)).

    Args:
        x: Input vector
        mean: Mean of the distribution (default 0, encouraging small weights
        var: Variance (default 1, for standard Gaussian prior)

    Returns:
        Log-density value
    """
    # @@ COMPLETE @@
    out = -0.5 * (jnp.sum((x - mean) ** 2) / var + len(x) * jnp.log(2 * jnp.
    return out
```

2. Random Walk with Metropolis-Hastings Correction

The Metropolis-Hastings (MH) algorithm is one of the most practical MCMC methods. It repeatedly:

1. **Proposes** a new parameter vector using a random walk
2. **Evaluates** the posterior probability at the new and current locations
3. **Accepts/Rejects** using the MH acceptance criterion

Key insights:

- MH automatically focuses sampling on high-posterior regions
- The step_size parameter controls proposal variance (tuning parameter)
- With symmetric proposal, acceptance ratio only depends on posterior ratio

For more details, check the lecture notes on MCMC theory.

1. Produces a sequence of samples – $\boldsymbol{w}_1, \boldsymbol{w}_2, \ldots, \boldsymbol{w}_s, \ldots$

- Imagine we've just produced $\boldsymbol{w}_{s-1}$
- MH firsts proposes a possible $\boldsymbol{w}_s$ (call it $\tilde{\boldsymbol{w}}_s$) based on $\boldsymbol{w}_{s-1}$.
- MH then decides whether or not to accept wfs
  - If accepted, $\boldsymbol{w}_s \leftarrow \tilde{\boldsymbol{w}}_s$
  - If not, $\boldsymbol{w}_s \leftarrow \boldsymbol{w}_{s-1}$

We need to treat $\tilde{\boldsymbol{w}}_s$ as a random variable conditioned on $\boldsymbol{w}_{s-1}$. We can choose whatever we like but a simple solution is to use a Gaussian centered on $\boldsymbol{w}_{s-1}$ with some covariance $\boldsymbol{\Sigma}_p$.

Regarding the acceptance, we need to compute the acceptance ratio. Check the lecture notes for the full derivation. The first thing that we need to compute is the un-normalized logposterior (i.e. the sum of loglikelihood and prior):

$$\log p(\boldsymbol{w}|\boldsymbol{X}, \boldsymbol{y}) \propto \log p(\boldsymbol{y}|\boldsymbol{w}, \boldsymbol{X}) + \log p(\boldsymbol{w}) := g(\boldsymbol{w}; \boldsymbol{X}, \boldsymbol{y})$$

Exercise: Complete the function below to compute the unnormalized log-posterior $g(\boldsymbol{w}; \boldsymbol{X}, \boldsymbol{y})$.

In [8]:
```
def posterior_logdensity(w, X, y):
    """Compute the unnormalized log-posterior (up to a constant).

    By Bayes' rule:
    log p(w|X,y) ∝ log p(y|X,w) + log p(w)
                 = (log-likelihood) + (log-prior)

    Args:
        w: Weight vector (parameters to optimize)
        X: Input features (N, D) array
        y: Binary labels (N,) array

    Returns:
        Log-posterior (unnormalized)
    """
    # @@ COMPLETE @@
    log_posterior = jnp.sum(bernoulli_logdensity(y, logistic(X @ w))) + gaus
    return log_posterior
```

Exercise: Now you can move to the actual random walk with MH. Complete the `mcmc_step()` function following the flowchart in the slides. Instead of using explicit `if` statements, you can use `jnp.where()` to make the code JIT-compilable.

```
# Python code
if condition:
    x = a
else:
```

```
    x = b
# Equivalent JAX code
x = jnp.where(condition, a, b)
```

In [9]:
```
def mcmc_step(w_prev, step_size, key, X, y):
    """Perform one step of Metropolis-Hastings MCMC for Bayesian logistic re

    MCMC Algorithm:
    1. PROPOSE: Suggest a new weight vector from a random walk
    2. EVALUATE: Compute posterior likelihood under old and new weights
    3. ACCEPT/REJECT: Use Metropolis-Hastings ratio to decide whether to mov


    Args:
        w_prev: Previous weight vector
        step_size: Standard deviation of proposal distribution (tuning param
        key: JAX random key for reproducibility
        X: Training features
        y: Training labels

    Returns:
        w_next: New weight vector (accepted or rejected)
    """
    key1, key2 = jax.random.split(key)  # Split key for two independent rand

    # @@ COMPLETE @@
    random_walk_step = jax.random.normal(key1, shape=w_prev.shape) * step_si
    w_proposed = w_prev + random_walk_step
    w_next = w_prev
    g_prev = posterior_logdensity(w_prev, X, y)
    g_proposed = posterior_logdensity(w_proposed, X, y)
    log_u = jax.random.uniform(key2)
    w_next = jnp.where(log_u < jnp.exp(g_proposed - g_prev), w_proposed, w_r
    return w_next
```

Below, run the sampling for multiple iterations.

Implementation Note: The code uses `jax.lax.scan()` for efficient loop execution. This is much faster than Python loops because:

1. It's JIT-compiled to machine code
2. Memory is allocated once, not repeatedly
3. JAX can parallelize operations across iterations

In [10]:
```
@partial(jax.jit, static_argnames=("n_samples"))
def run_sampling(w_init, key_init, n_samples, step_size):
    """Run MCMC sampling for Bayesian logistic regression.

    This function repeatedly calls mcmc_step() to build a sequence of sample
    The @jax.jit decorator compiles this to machine code for speed.

    The jax.lax.scan() is JAX's efficient way to implement loops with state:
    - It's faster than Python for loops
    - It's more memory-efficient when JIT-compiled
```

```
    - It maintains the Markov chain state w across iterations

    Args:
        w_init: Initial weight vector
        key_init: JAX random key for reproducibility
        n_samples: Number of MCMC iterations to run
        step_size: Proposal step size (affects acceptance rate)

    Returns:
        samples: Array of shape (n_samples, n_params) containing the MCMC ch
    """
    w = jnp.atleast_1d(w_init)
    keys = jax.random.split(
        key_init, n_samples
    )  # Create independent keys for each step

    # Use jax.lax.scan for efficient sequential computation:
    # - lambda w, key: ... defines one step of the loop
    # - (mcmc_step(...), w) means: compute new state, record current state
    # - [1] extracts the recorded states (the chain)
    samples = jax.lax.scan(
        lambda w, key: (mcmc_step(w, step_size, key, X, y), w), w, keys
    )[1]
    return samples
```

Note: The code above uses JAX's powerful features to dramatically speed up the sampling process. You are free to re-implement the sampling loop using standard Python loops and check how much slower it is.

Exercise: Run the sampler for 5000 steps (you can fix the step size for the proposal to 0.5).

In [12]:
```
w_init = np.zeros(2)
key_init = jax.random.key(0)

# @@ COMPLETE @@
samples = run_sampling(w_init, key_init, n_samples=1000, step_size=0.5)
print("Shape of samples:", samples.shape)
```

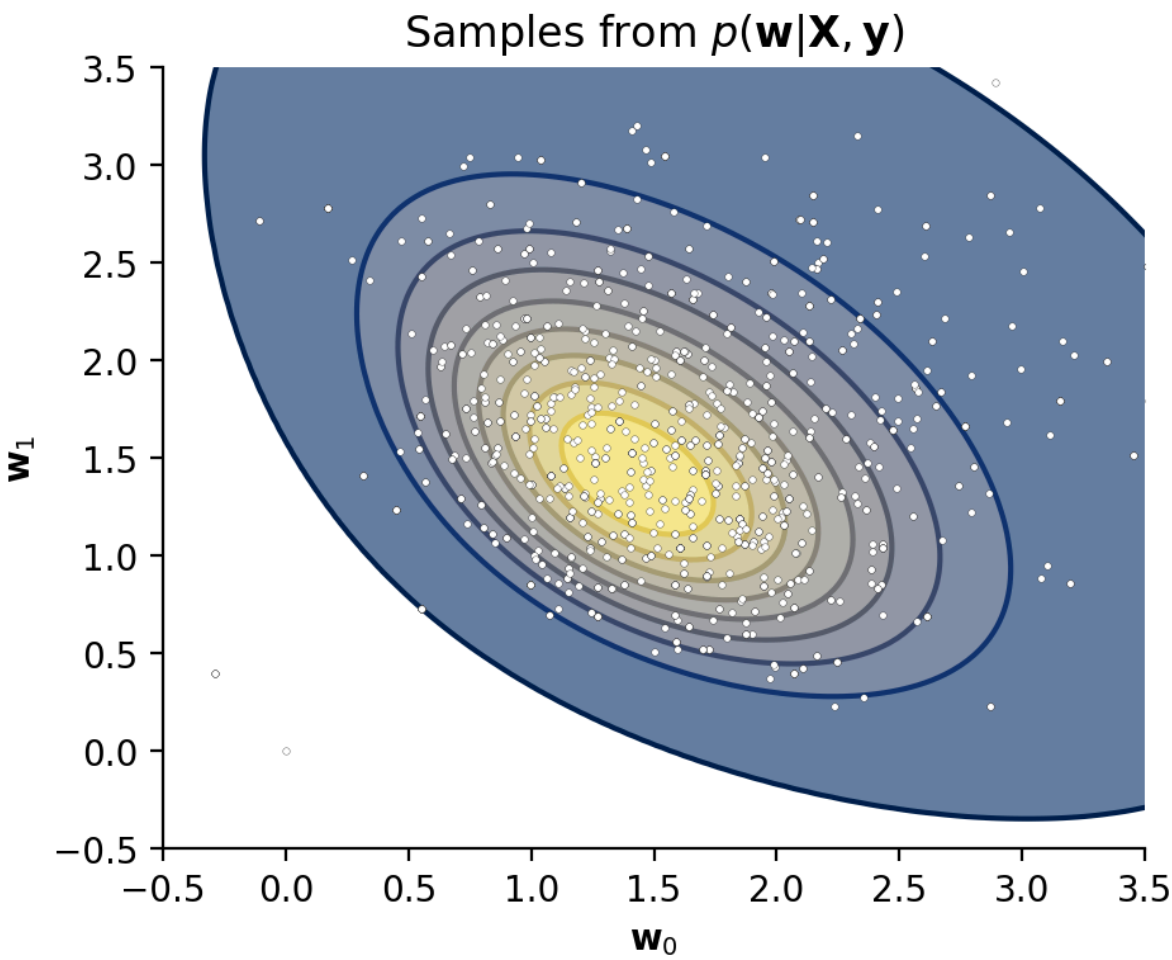
```
Shape of samples: (1000, 2)
```

Exercise: Plot the samples and their distribution.

In [13]:
```
def plot_posterior(ax):
    """Visualize the posterior distribution over the weight space."""
    xx, yy, X_plot = get_grid(xlim=(-1.5, 5.5), N=250)
    # Use vmap to evaluate posterior at all grid points efficiently
    vals = jnp.exp(
        jax.vmap(posterior_logdensity, in_axes=(0, None, None))(X_plot, X, y
    )
    levels = np.linspace(1e-5, max(vals), 10)
    patch = ax.contourf(
        xx, yy, vals.reshape(*xx.shape), cmap="cividis", alpha=0.6, levels=l
```

```
    )
    ax.contour(xx, yy, vals.reshape(*xx.shape), cmap="cividis", levels=patch
```

In [14]:
```
fig, ax = plt.subplots(figsize=[5, 4])
ax.scatter(
    *samples.T, s=4, edgecolor="black", linewidth=0.1, zorder=10, facecolor=
)
plot_posterior(ax)
ax.set_xlim(-0.5, 3.5)
ax.set_ylim(-0.5, 3.5)
ax.set_xlabel(r"$\mathbf{w}_0$")
ax.set_ylabel(r"$\mathbf{w}_1$")
ax.set_title(r"Samples from $p(\mathbf{w}|\mathbf{X},\mathbf{y}) $")
plt.show()
```



Why Bayesian Classification?

Traditional ML gives point estimates: a single weight vector w. Bayesian ML gives a distribution over weights: $p(w|X, y)$.

This distribution represents our uncertainty about the weights. When making predictions:

- Instead of using one w, we average predictions over the distribution

- This gives calibrated probability estimates (properly quantified uncertainty)
- Better for decision-making when stakes are high

MCMC gives us samples from this posterior distribution. This is possible by computing the following expectation:

$$P(y_{\text{new}} = 1 | \boldsymbol{x}_{\text{new}}, \boldsymbol{X}, \boldsymbol{y}) = \mathbb{E}_{p(\boldsymbol{w}|\boldsymbol{X},\boldsymbol{y},\sigma_{\text{n}})} h(\boldsymbol{w}^\top \boldsymbol{x}_{\text{new}}) = \int h(\boldsymbol{w}^\top \boldsymbol{x}_{\text{new}}) p(\boldsymbol{w}|\boldsymbol{X}, \boldsymbol{y}) \mathrm{d}\boldsymbol{w}$$

Exercise: Complete the next function to compute this expectation. And compute the probability $P(y_{\text{new}} = 1 | \boldsymbol{x}_{\text{new}}, \boldsymbol{X}, \boldsymbol{y})$ when $\boldsymbol{x}_{\text{new}} = [2, -4]^\top$. **Note:**

In [15]:
```
def predict_mcmc(x_new, samples):
    """Make a Bayesian prediction by averaging over the posterior.

    Args:
        x_new: Single input vector (D,)
        samples: MCMC samples of shape (n_samples, n_params)

    Returns:
        Predicted probability that y=1 for input x_new
    """
    # @@ COMPLETE @@
    p_y_new = logistic(samples @ x_new)  # Shape (n_samples,)
    probs = jnp.mean(p_y_new)  # Average over samples to get predictive prob
    return probs


x_new = jnp.array([2.0, -4.0])
# @@ COMPLETE @@
prob = predict_mcmc(x_new, samples)
print(f"P(y_new = 1 | x_new, X, y) = {prob:.4f}")
```

```
P(y_new = 1 | x_new, X, y) = 0.1672
```

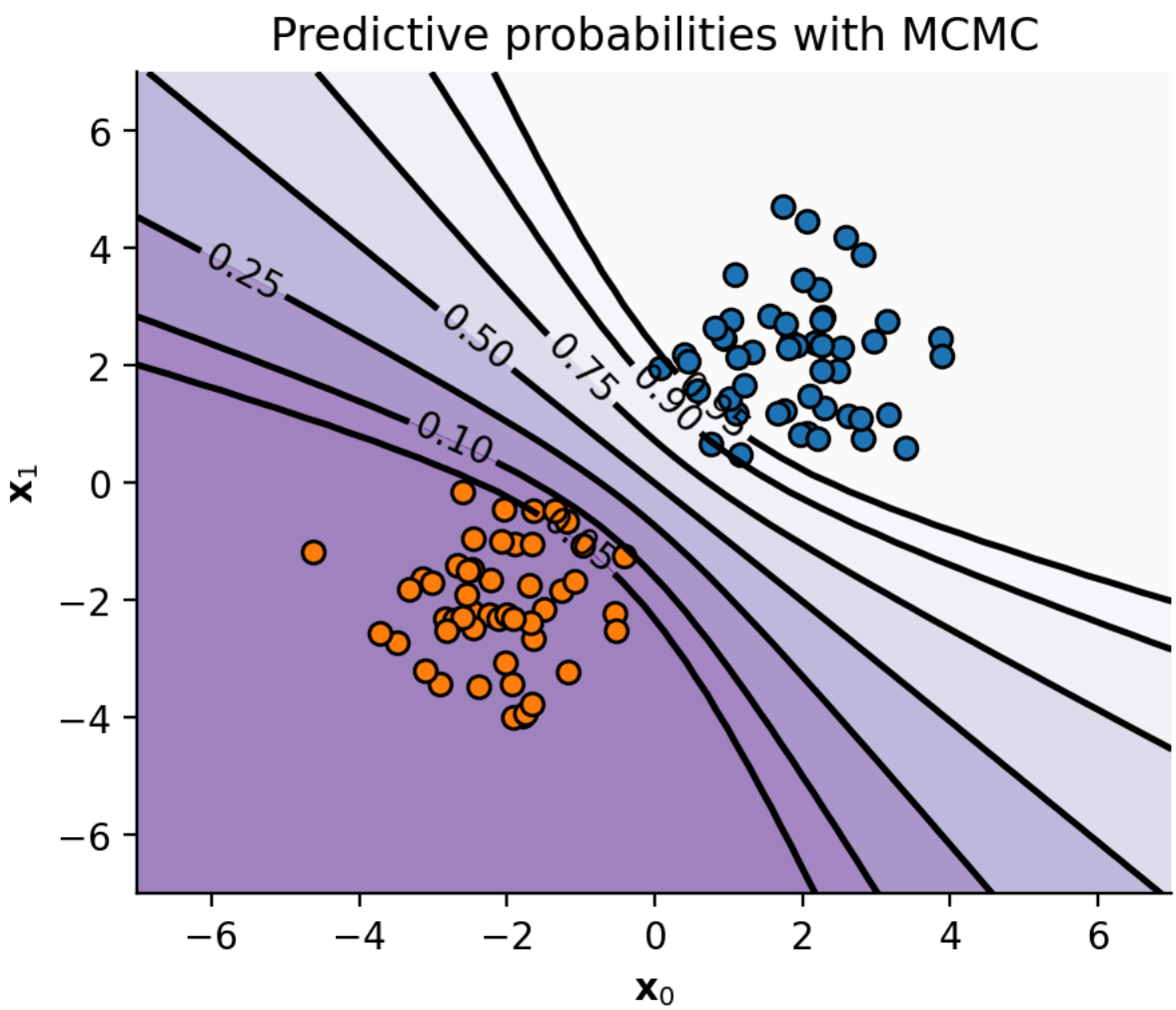
Let's also vectorize the prediction function to predict on a batch of inputs. Make sure to understand the syntax below and how it works.

In [16]:
```
predict_mcmc_batch = jax.jit(jax.vmap(predict_mcmc, in_axes=(0, None)))
```

Exercise: Plot the predictive probabilities over a grid of inputs. Use the `predict_mcmc_batch()` function defined above.

In [17]:
```
def plot_decision_boundary(xx, yy, P, ax):
    """Visualize predictive probabilities as a contour map."""
    P = P.reshape(*xx.shape)  # Reshape flat predictions back to 2D grid
    levels = [0, 0.05, 0.1, 0.25, 0.5, 0.75, 0.9, 0.95, 1]  # Probability co
    cs = ax.contour(xx, yy, P, levels, colors="k", linewidths=1.8, zorder=10
    ax.clabel(cs, inline=1, fontsize=10)  # Label contours with probability
    cs = ax.contourf(xx, yy, P, levels, cmap="Purples_r", alpha=0.5)


xx, yy, grid = get_grid((-7, 7), N=50)
```

```
probs = predict_mcmc_batch(grid, samples)
fig, ax = plt.subplots(figsize=[5, 4])
plot_decision_boundary(xx, yy, probs, ax)
plot_data(X, y, ax)

ax.set_title("Predictive probabilities with MCMC")
ax.set_xlabel(r"$\mathbf{x}_0$")
ax.set_ylabel(r"$\mathbf{x}_1$")
plt.show()
```



Convergence Diagnostics: Burn-in and Mixing

After running the sampler, we must verify convergence before using the samples. Two critical concepts:

1. **Burn-in:** Early samples may be far from the posterior because we start at an arbitrary point. Discard these initial iterations (e.g., first 10-20% of samples).

2. **Mixing:** A good chain should explore the entire posterior efficiently.

    - Poor mixing: Chain gets stuck, moves slowly, shows strong autocorrelation
    - Good mixing: Chain explores freely, high acceptance rate, low autocorrelation

The step_size parameter is crucial for mixing:

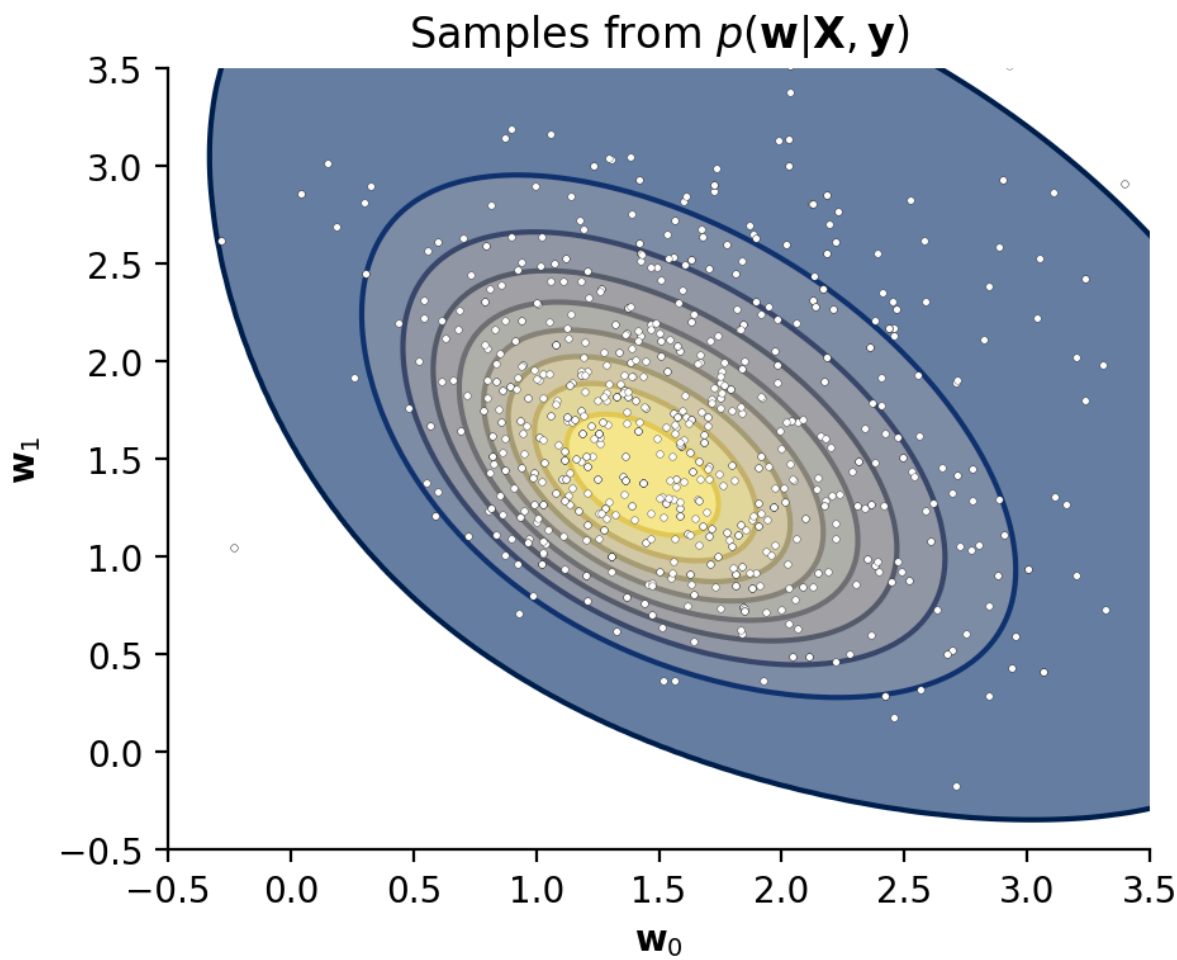
- Too small: High acceptance, but moves slowly (poor mixing)
- Too large: Low acceptance, chain rejects proposals (stuck)
- Just right: ~50% acceptance rate for MH (typical rule of thumb)

Exercise: Sometimes choosing the initial point to start the MCMC is not easy. Choose a starting point very far way from the posterior (say $[-4, -4]^\top$) and run the sampler for 10000 (set the step size to 0.1). Plot the trajectory of the first 200 steps with a different color. What do you observe?

In [20]:
```
# @@ COMPLETE @@
key_init = jax.random.key(0)
w_init = jnp.array([-4.0, -4.0])
samples = run_sampling(w_init, key_init, n_samples=1000, step_size=0.5)

print("Shape of samples:", samples.shape)
```

```
Shape of samples: (1000, 2)
```

In [21]:
```
fig, ax = plt.subplots(figsize=[5, 4])
# @@ COMPLETE @@
ax.scatter(
    *samples.T, s=4, edgecolor="black", linewidth=0.1, zorder=10, facecolor=
)
plot_posterior(ax)
ax.set_xlim(-0.5, 3.5)
ax.set_ylim(-0.5, 3.5)

ax.set_xlabel(r"$\mathbf{w}_0$")
ax.set_ylabel(r"$\mathbf{w}_1$")
ax.set_title(r"Samples from $p(\mathbf{w}|\mathbf{X},\mathbf{y}) $")
plt.show()
```



This effect can be mitigated by using burn-in. Burn-in is intended to give the Markov Chain time to reach its equilibrium distribution, particularly if it has started from a lousy starting point. To "burn in" a chain, you just discard the first n samples before you start collecting points.

The idea is that a "bad" starting point may over-sample regions that are actually very low probability under the equilibrium distribution before it settles into the equilibrium distribution. If you throw those points away, then the points which should be unlikely will be suitably rare.

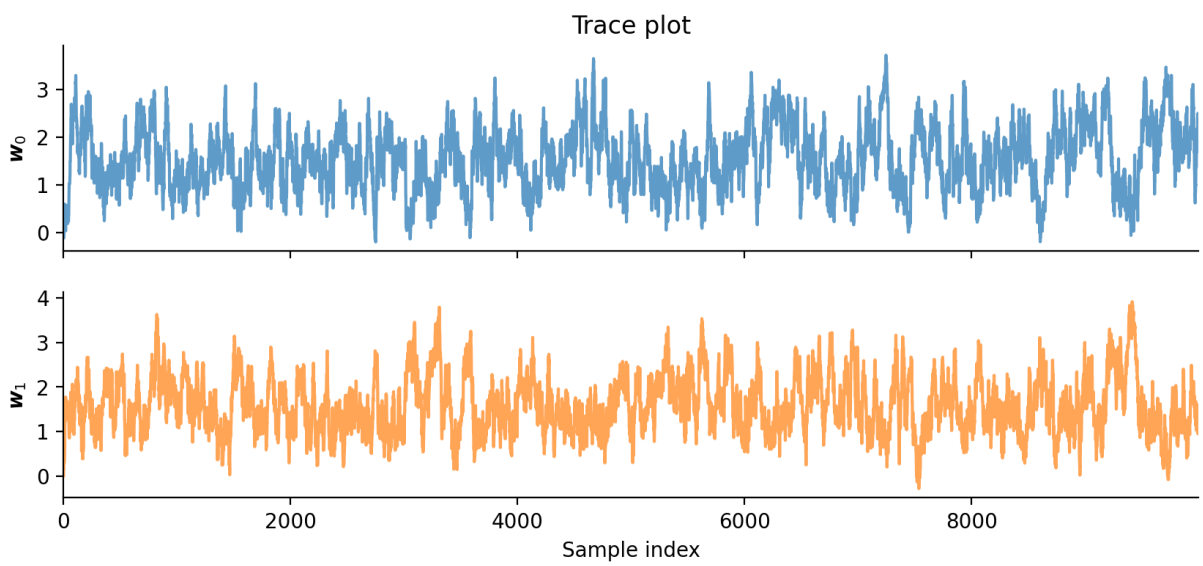
It clear that the burn-in is more of a hack/artform than a principled technique. In theory, you could just sample for a really long time or find some way to choose a decent starting point instead.

Trace plots

The trace plot shows the sampled values of a parameter over time (iterations). This plot helps you to judge how quickly the MCMC procedure converges in distribution—that is, how quickly it forgets its starting values.

Exercise: For the two parameters of $\boldsymbol{w}$, plot their trace.

In [28]:
```
w_init = [0.0, 0.0]
samples = run_sampling(w_init, key_init, 10000, 0.2)

fig, (ax0, ax1) = plt.subplots(2, 1, figsize=[10, 4], sharex=True)

# @@ COMPLETE @@
ax0.plot(samples[:, 0], color="tab:blue", alpha=0.7)
ax1.plot(samples[:, 1], color="tab:orange", alpha=0.7)

ax0.set_ylabel(r"$\boldsymbol{w}_0$")
ax1.set_ylabel(r"$\boldsymbol{w}_1$")
ax0.set_title("Trace plot")
ax1.set_xlabel(r"Sample index")
plt.show()
```

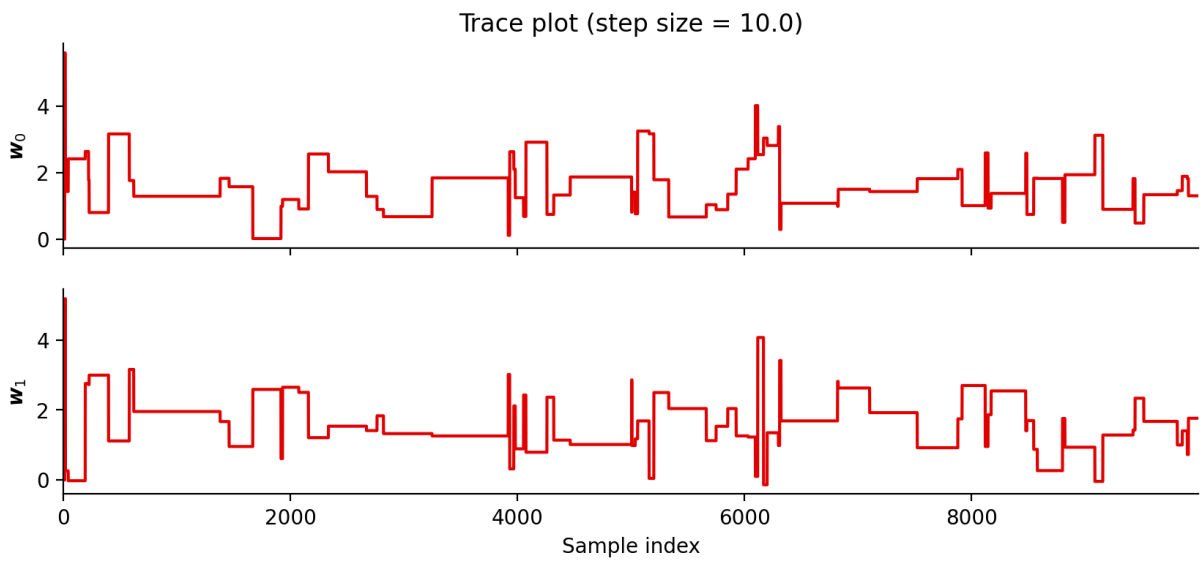


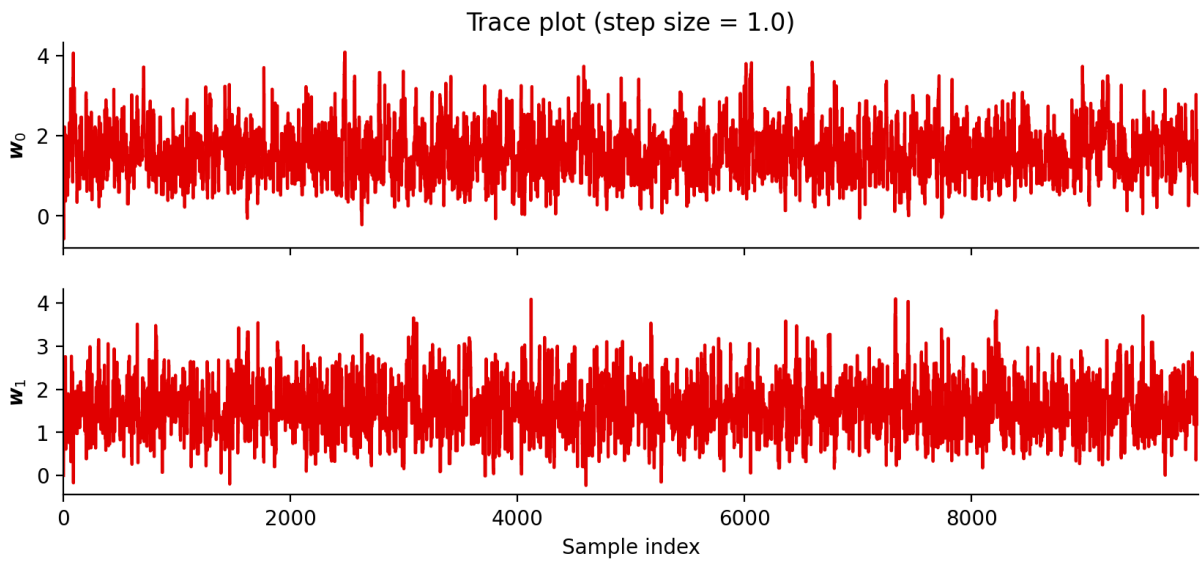
Exercise: Change the step size, sample and plot the trace. Try 10, 1, 0.1 and 0.01. What behavior do you see? Comment the plots.

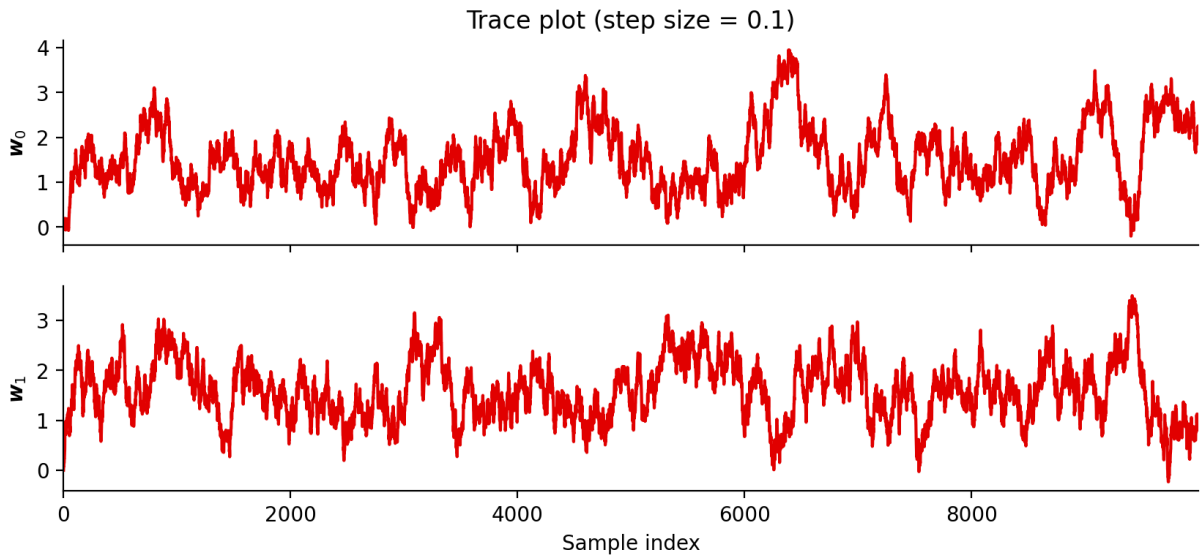
The trace plot with differnet step sizes should show how the choice of step size affects the convergence and mixing of the MCMC sampler. With a very large step size (e.g., 10), you may see erratic jumps and low acceptance rates, indicating poor mixing. With a very small step size (e.g., 0.01), the trace may show slow movement and high autocorrelation, indicating that the chain is not exploring the posterior efficiently. An intermediate step size (e.g., 0.1) should show a more stable trace with good mixing and convergence towards the posterior distribution.

In [27]:
```
# @@ COMPLETE @@

for step_size in [10.0, 1.0, 0.1, 0.01]:
    samples = run_sampling(w_init, key_init, 10000, step_size)

    fig, (ax0, ax1) = plt.subplots(2, 1, figsize=[10, 4], sharex=True)
```

```python
    ax0.plot(samples[:, 0], color="xkcd:red")
    ax1.plot(samples[:, 1], color="xkcd:red")

    ax0.set_ylabel(r"$\boldsymbol{w}_0$")
    ax1.set_ylabel(r"$\boldsymbol{w}_1$")
    ax0.set_title(f"Trace plot (step size = {step_size})")
    ax1.set_xlabel(r"Sample index")
    plt.show()
```







3.2 Multiple chains and a more sophisticated diagnostics

In an attempt to assuage concerns of poor convergence, we typically run multiple independent chains to see if the obtained distribution is similar across chains. We can also visually inspect the sample paths of the chains via trace plots as well as study summary statistics such as the empirical autocorrelation function.

Exercise: Complete the following function to run multiple chain.

In [ ]:
```
@partial(jax.jit, static_argnames=("n_samples", "n_chains"))
def run_multiple_chains(w_init, key_init, n_samples, step_size, n_chains):
    """Run multiple independent MCMC chains in parallel.

    Running multiple chains helps us:
    1. Diagnose convergence: Do all chains explore the same posterior regior
    2. Increase effective sample size: Combine samples from all chains
    3. Detect multimodality: Different chains finding different modes
```

```
    The jax.vmap() applies run_sampling over n_chains independent chains
    with different random seeds, running them all at once for efficiency.

    Args:
        w_init: Initial weight vector (same for all chains)
        key_init: JAX random key (will be split into n_chains keys)
        n_samples: Number of iterations per chain
        step_size: Proposal step size
        n_chains: Number of independent chains to run

    Returns:
        chains: Array of shape (n_chains, n_samples, n_params)
    """
    keys = jax.random.split(
        key_init, n_chains
    )

    chains = jax.vmap(run_sampling, in_axes=(None, 0, None, None))(
        w_init, keys, n_samples, step_size
    )
    return chains
```

In [30]:
```
chains = run_multiple_chains(w_init, key_init, 5000, 0.2, 4)
print("Shape of chains:", chains.shape)
```

```
Shape of chains: (4, 5000, 2)
```

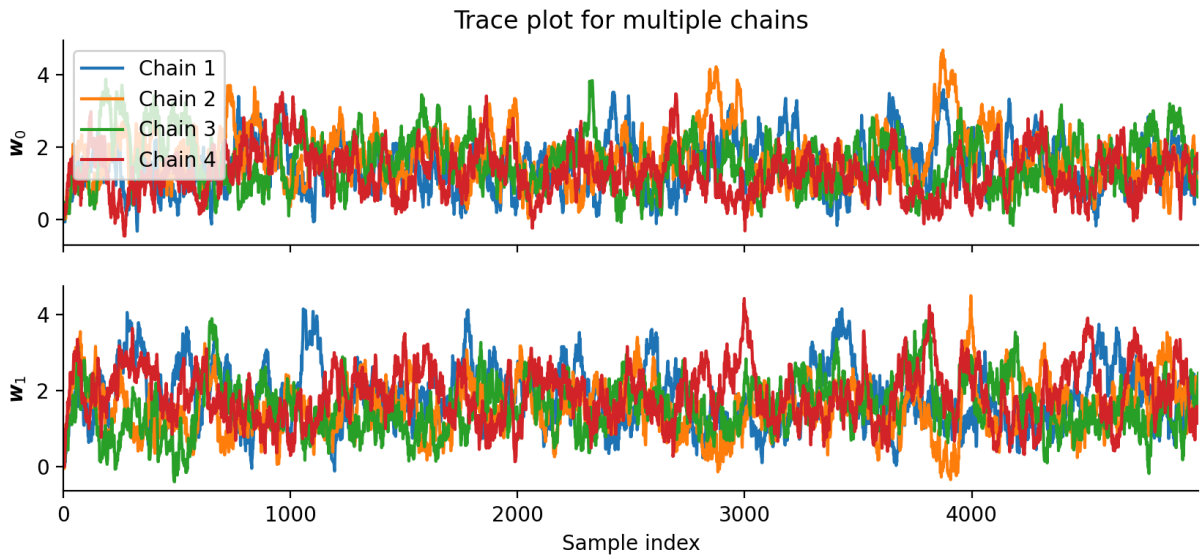
Note: The shape of `chains` should be `(n_chains, n_samples, n_params)`.
Implementation note: The code above uses `jax.vmap` to vectorize the sampling over multiple chains. This is more efficient than using a Python loop, as it allows JAX to optimize the computation across all chains simultaneously. The equivalent code using a Python loop would be:

```
chains = []
for i in range(n_chains):
    chain = run_sampling(w_init, key, n_samples, step_size)
    chains.append(chain)
```

The for-loop version is easier to read but significantly slower.

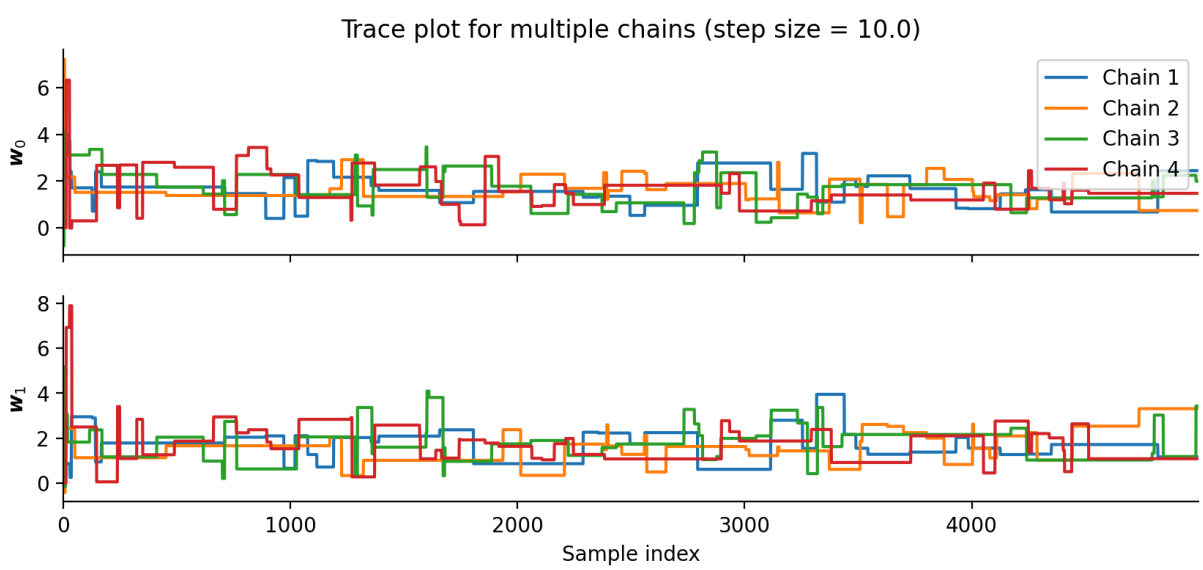
Exercise: Plot the trace plots for the multiple chains.

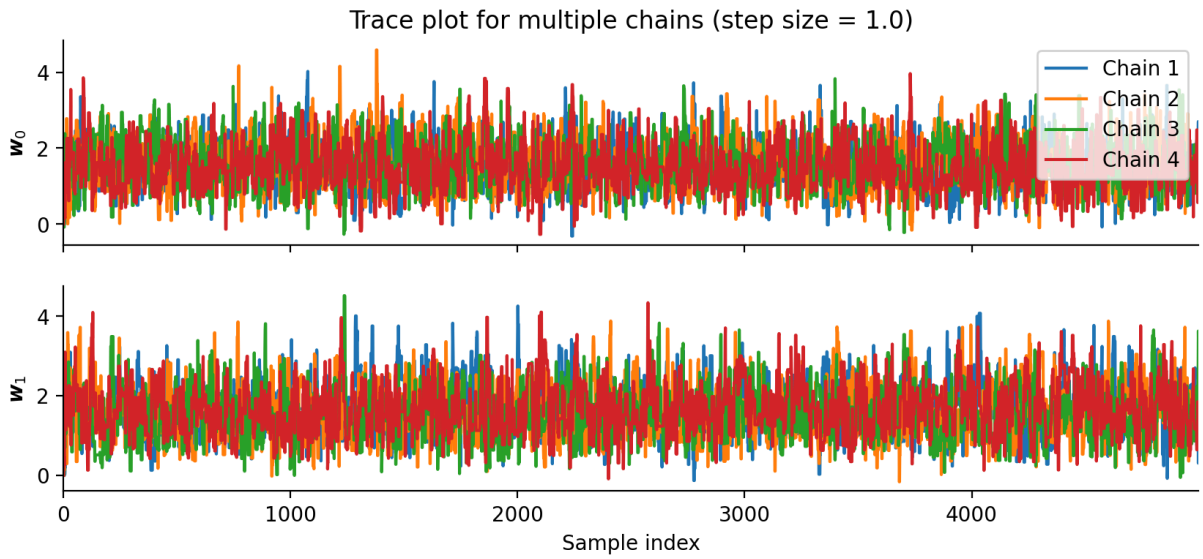
In [31]:
```
fig, axes = plt.subplots(2, 1, figsize=[10, 4], sharex=True)
for i in range(chains.shape[0]):
    axes[0].plot(chains[i, :, 0], label=f"Chain {i + 1}")
    axes[1].plot(chains[i, :, 1], label=f"Chain {i + 1}")
axes[0].set_ylabel(r"$\boldsymbol{w}_0$")
axes[1].set_ylabel(r"$\boldsymbol{w}_1$")
axes[0].set_title("Trace plot for multiple chains")
axes[1].set_xlabel(r"Sample index")
axes[0].legend()
plt.show()
```

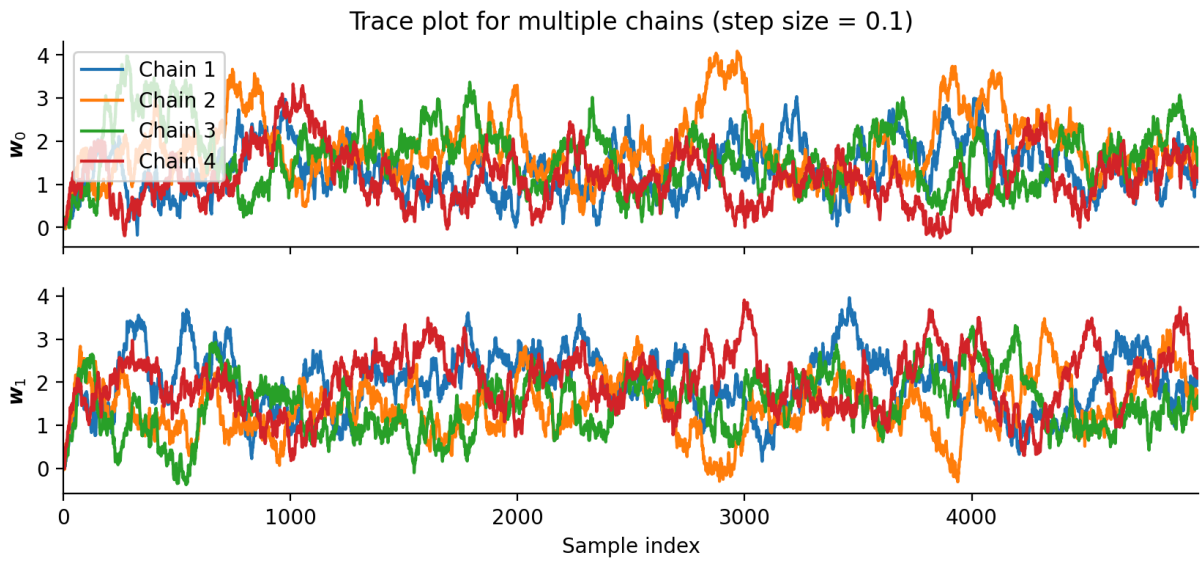


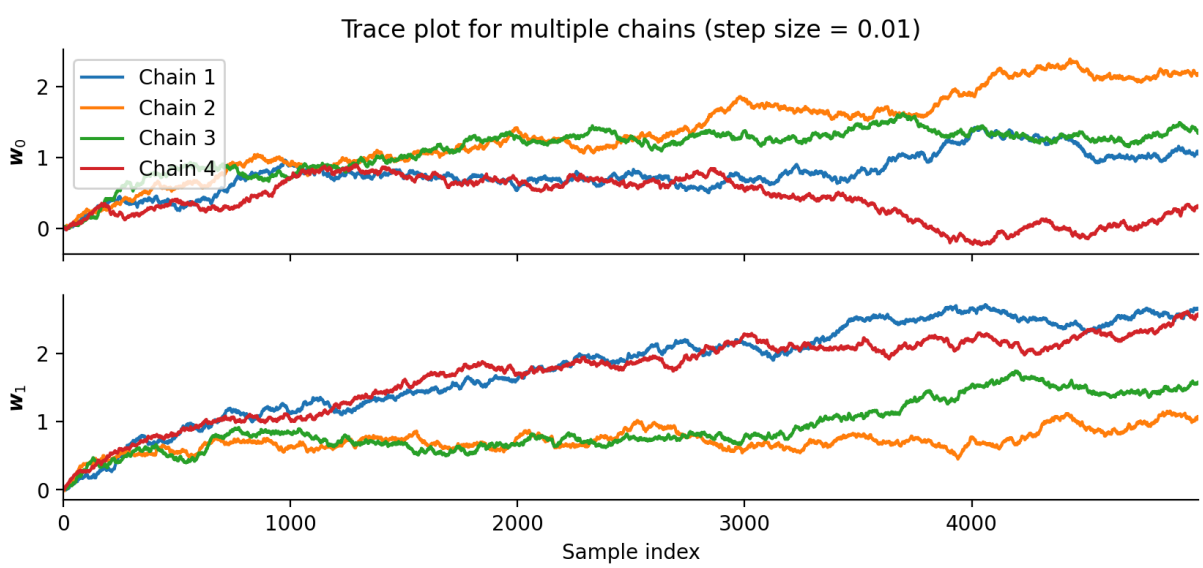
Exercise: Try to change the step size (try 10, 1, 0.1 and 0.01) and see how visually the chains behave. What do you observe?

In [ ]:
```
for step_size in [10.0, 1.0, 0.1, 0.01]:
    # @@ COMPLETE @@

    chains = run_multiple_chains(w_init, key_init, 5000, step_size, 4)

    fig, axes = plt.subplots(2, 1, figsize=[10, 4], sharex=True)
    for i in range(chains.shape[0]):
        axes[0].plot(chains[i, :, 0], label=f"Chain {i + 1}")
        axes[1].plot(chains[i, :, 1], label=f"Chain {i + 1}")
    axes[0].set_ylabel(r"$\boldsymbol{w}_0$")
    axes[1].set_ylabel(r"$\boldsymbol{w}_1$")
    axes[0].set_title(f"Trace plot for multiple chains (step size = {step_si
    axes[1].set_xlabel(r"Sample index")
    axes[0].legend()
    plt.show()
```









When testing with multiple chains, you may observe that with a very large step size (e.g., 10), the chains may show erratic behavior and fail to converge to the same distribution, indicating poor mixing. With a very small step size (e.g., 0.01), the chains may move very

slowly and show high autocorrelation, suggesting that they are not exploring the posterior efficiently. An intermediate step size (e.g., 0.1) should show more consistent behavior across chains, with better convergence and mixing towards the posterior distribution.

NumPyro Implementation

NumPyro is a probabilistic programming library built on JAX that allows for flexible and efficient Bayesian inference. It provides high-level abstractions for defining probabilistic models and performing inference using MCMC (and VI).

In the next section, you will find an implementation of the same Bayesian logistic regression model using NumPyro's MCMC capabilities. Use this to compare and contrast with your manual implementation above. Do you observe similar results? If not, check your implementation above for possible mistakes.

In [33]:
```
try:
    import numpyro
except ImportError:
    import os

    print("Installing numpyro...")
    os.system("pip install numpyro")

import numpyro
import numpyro.distributions as dist
```

```
Installing numpyro...
Collecting numpyro
  Downloading numpyro-0.20.1-py3-none-any.whl.metadata (35 kB)
Requirement already satisfied: jax>=0.7.0 in /Users/torstein/Desktop/Skole/S
tudie/8.Semester/ASI/Lab/asi-labs/.venv-lab2/lib/python3.14/site-packages (f
rom numpyro) (0.9.2)
Requirement already satisfied: jaxlib>=0.7.0 in /Users/torstein/Desktop/Skol
e/Studie/8.Semester/ASI/Lab/asi-labs/.venv-lab2/lib/python3.14/site-packages
(from numpyro) (0.9.2)
Collecting multipledispatch (from numpyro)
  Downloading multipledispatch-1.0.0-py3-none-any.whl.metadata (3.8 kB)
Requirement already satisfied: numpy in /Users/torstein/Desktop/Skole/Studi
e/8.Semester/ASI/Lab/asi-labs/.venv-lab2/lib/python3.14/site-packages (from
numpyro) (2.4.4)
Collecting tqdm (from numpyro)
  Downloading tqdm-4.67.3-py3-none-any.whl.metadata (57 kB)
Requirement already satisfied: ml_dtypes>=0.5.0 in /Users/torstein/Desktop/S
kole/Studie/8.Semester/ASI/Lab/asi-labs/.venv-lab2/lib/python3.14/site-packa
ges (from jax>=0.7.0->numpyro) (0.5.4)
Requirement already satisfied: opt_einsum in /Users/torstein/Desktop/Skole/S
tudie/8.Semester/ASI/Lab/asi-labs/.venv-lab2/lib/python3.14/site-packages (f
rom jax>=0.7.0->numpyro) (3.4.0)
Requirement already satisfied: scipy>=1.13 in /Users/torstein/Desktop/Skole/
Studie/8.Semester/ASI/Lab/asi-labs/.venv-lab2/lib/python3.14/site-packages
(from jax>=0.7.0->numpyro) (1.17.1)
Downloading numpyro-0.20.1-py3-none-any.whl (388 kB)
Downloading multipledispatch-1.0.0-py3-none-any.whl (12 kB)
Downloading tqdm-4.67.3-py3-none-any.whl (78 kB)
Installing collected packages: multipledispatch, tqdm, numpyro
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 3/3 [numpyro]m2/3 [numpyro]
Successfully installed multipledispatch-1.0.0 numpyro-0.20.1 tqdm-4.67.3
```

What is Probabilistic Programming?

Probabilistic programming lets you write down your statistical model directly in code, rather than manually implementing the likelihood and MCMC sampling logic (like we did above). The framework handles all the inference automatically.

A model in probabilistic programming is a function that describes the joint distribution of your data and parameters using random variables.

Remember that in our case, the joint distribution is:

$$p(\boldsymbol{y}, \boldsymbol{w}|\boldsymbol{X}) = \prod_{i=1}^{N} p(y_i|\boldsymbol{x}_i, \boldsymbol{w})p(\boldsymbol{w})$$

where:

- $p(y_i|\boldsymbol{x}_i, \boldsymbol{w})$ is the Bernoulli likelihood, with $p_i = h(\boldsymbol{w}^\top \boldsymbol{x}_i)$
- $p(\boldsymbol{w})$ is the Gaussian prior

The next cell defines this model in NumPyro.

In [34]:
```
def model(X, y):
    """Define the Bayesian logistic regression model in NumPyro.

    This function specifies the prior and likelihood for the model.
    NumPyro uses this to construct the posterior distribution.

    Args:
        X: Input features (N, D) array
        y: Binary labels (N,) array
    """
    N, d = X.shape

    # Prior over weights: w ~ N(0, I)
    w = numpyro.sample("w", dist.Normal(jnp.zeros(d), jnp.ones(d)))

    # Probs are a deterministic transformation of inputs and weights: p = si
    probs = numpyro.deterministic("probs", logistic(X @ w))

    # Likelihood over the data: y_i ~ Bernoulli(probs_i)
    # Put the i.i.d. assumption using the plate notation
    # This essentially means we have N independent samples from the likelihc
    with numpyro.plate("data", N):
        numpyro.sample("obs", dist.Bernoulli(probs=probs), obs=y)
```

Understanding the NumPyro Model

Breaking Down the Model Line-by-Line:

1. `w = numpyro.sample("w", dist.Normal(jnp.zeros(d), jnp.ones(d)))`
   - This declares that `w` is a **random variable** sampled from a Normal distribution
   - Name `"w"`: Labels this variable so NumPyro can track it
   - `dist.Normal(0, 1)`: The prior distribution (mean=0, std=1)
   - This is equivalent to our manual `gaussian_logdensity(w)` but declarative
   - In this case, sample does **not** mean "draw a sample now" – it means "this is a random variable" that follows this (unconditioned) prior distribution
2. `probs = numpyro.deterministic("probs", logistic(X @ w))`
   - Declares a **deterministic** transformation (not random, just a computation)
   - Computes: `probs = logistic(X @ w)` (same as our manual implementation)
   - The name allows us to extract and inspect these values after inference
3. `with numpyro.plate("data", N):`
   - A "plate" represents **repeated independent samples** (all N data points)
   - This is the way to implement the i.i.d. assumption in probabilistic programming

$$p(y|X, w) = \prod_{i=1}^{N} p(y_i|x_i, w)$$

- This is the same as summing log-likelihoods over all data points in our manual code ()

4. **`numpyro.sample("obs", dist.Bernoulli(probs=probs), obs=y)`**

   - Declares the likelihood: each `y[i] ~ Bernoulli(p[i])` where `p[i] = sigmoid(logits[i])`
   - `obs=y` : The key difference from above! This **conditions** the model on observed data
   - Tells NumPyro: "The data is fixed at `y`, it's not an unconditioned random variable anymore"
   - Equivalent to our manual `bernoulli_logdensity(y, p)`

Now that we have defined the model, we can use NumPyro's implementation of MCMC to sample from the posterior distribution. We will use the Hamiltonian Monte Carlo (HMC) algorithm (see lecture notes for details), because MH is too basic and not implemented in NumPyro.

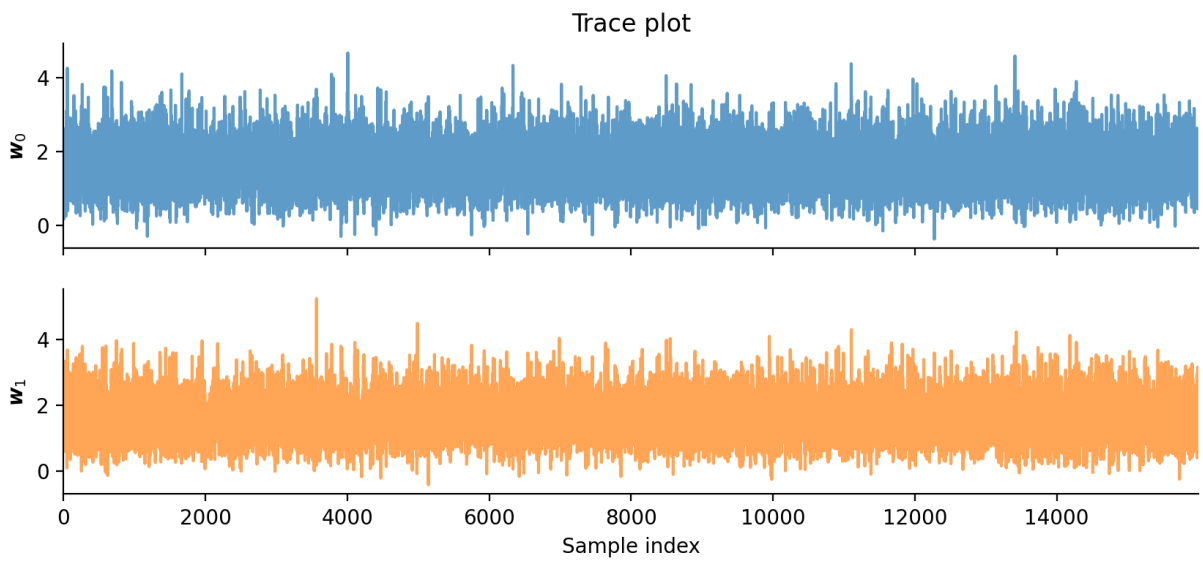
In [35]:
```
from numpyro.infer import MCMC, HMC

inference = MCMC(
    HMC(model, step_size=0.1, adapt_step_size=False),
    num_warmup=1000,
    num_samples=4000,
    num_chains=4,
)

inference.run(jax.random.key(0), X=X, y=y)

print(inference.print_summary())
```

```
sample: 100%|██████████| 5000/5000 [00:00<00:00, 5235.31it/s, 63 steps of si
ze 1.00e-01. acc. prob=1.00]
sample: 100%|██████████| 5000/5000 [00:00<00:00, 6945.38it/s, 63 steps of si
ze 1.00e-01. acc. prob=1.00]
sample: 100%|██████████| 5000/5000 [00:00<00:00, 6854.22it/s, 63 steps of si
ze 1.00e-01. acc. prob=1.00]
sample: 100%|██████████| 5000/5000 [00:00<00:00, 6828.87it/s, 63 steps of si
ze 1.00e-01. acc. prob=1.00]
                mean       std    median      5.0%     95.0%     n_eff     r
_hat
      w[0]      1.58      0.64      1.53      0.53      2.60   8205.84
1.00
      w[1]      1.58      0.65      1.54      0.50      2.59   7595.18
1.00

Number of divergences: 0
None
```

In [37]:
```
samples = inference.get_samples()["w"]
print("Shape of samples:", samples.shape)
```

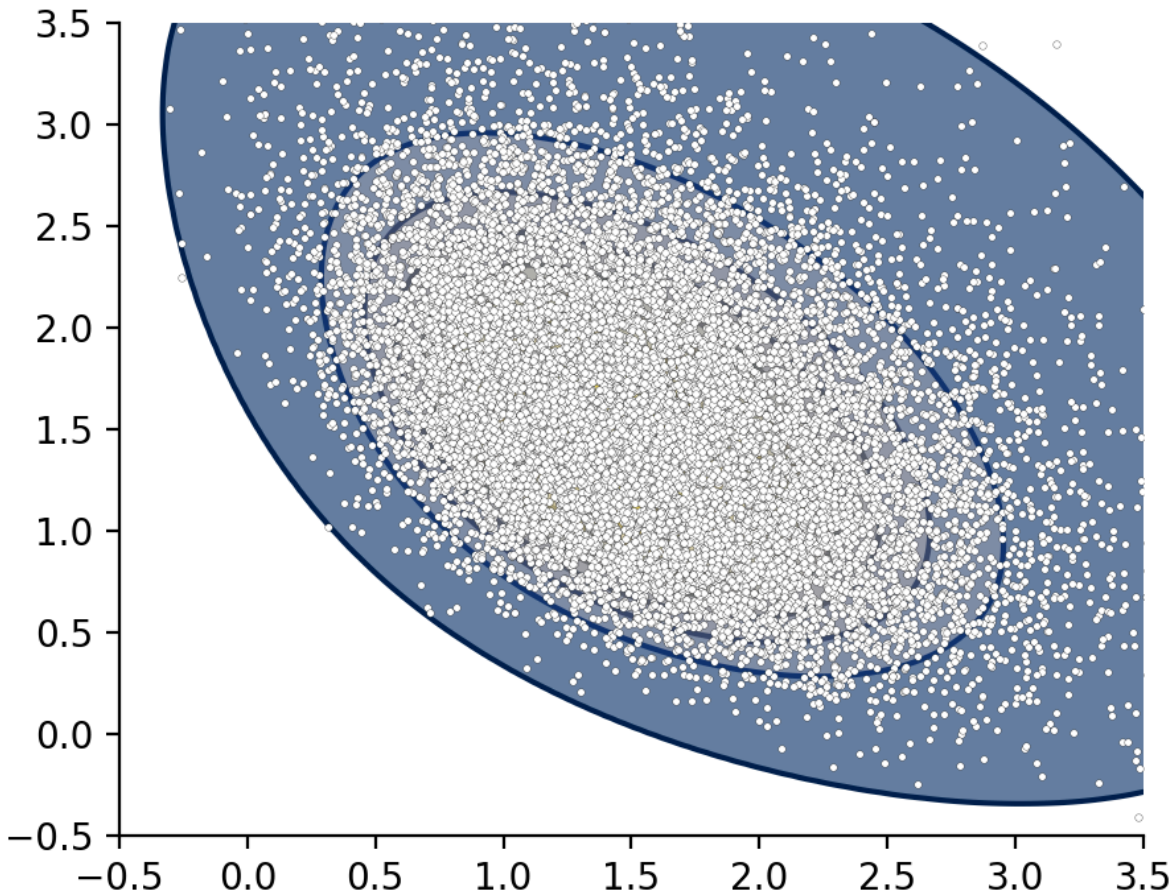
```
fig, (ax0, ax1) = plt.subplots(2, 1, figsize=[10, 4], sharex=True)

# @@ COMPLETE @@
ax0.plot(samples[:, 0], color="tab:blue", alpha=0.7)
ax1.plot(samples[:, 1], color="tab:orange", alpha=0.7)
ax0.set_ylabel(r"$\boldsymbol{w}_0$")
ax1.set_ylabel(r"$\boldsymbol{w}_1$")
ax0.set_title("Trace plot")
ax1.set_xlabel(r"Sample index")
plt.show()
```

```
Shape of samples: (16000, 2)
```



In [38]:
```
fig, ax = plt.subplots(figsize=[5, 4])
ax.scatter(
    *samples.T, s=4, edgecolor="black", linewidth=0.1, zorder=10, facecolor=
)
plot_posterior(ax)
ax.set_xlim(-0.5, 3.5)
ax.set_ylim(-0.5, 3.5)
```

Out[38]:
```
(-0.5, 3.5)
```



Making predictions with NumPyro samples is the same as before, but you can also use NumPyro's built-in predictive utilities for convenience.

In [39]:
```
from numpyro.infer import Predictive

predictive = Predictive(model, posterior_samples=inference.get_samples())

x_new = jnp.array([[2.0, -4.0]])
predictions = predictive(
    jax.random.key(1), X=x_new, y=None
)  # y=None since we want to predict
probs = jnp.mean(predictions["obs"], axis=0)
print(f"P(y_new = 1 | x_new, X, y) = {probs[0]:.4f}")
```

```
P(y_new = 1 | x_new, X, y) = 0.2002
```

The value above should be similar to what you obtained with your manual MCMC implementation. This confirms that both implementations are consistent and correctly capture the posterior distribution over the weights in Bayesian logistic regression. If there are discrepancies, review your manual code for potential errors.

You can also try to visualize the decision boundary using the NumPyro predictive samples, similar to what we did before. It should yield comparable results.